Programmable Matter 4 Report

Ibrahim Ali Khan
Universität Paderborn, Germany

Ahmet Can Kaytaz
Universität Paderborn, Germany

Pooja Garg
Universität Paderborn, Germany

Rasika Jayavelu
Universität Paderborn, Germany

Thevin Arokiaraj Lawrence
Universität Paderborn, Germany

Abstract

We present a modular, distributed pipeline for testing rotational and reflective symmetries in the geometric amoebot model. Particles live on a hexagonal grid, communicate via short "beeps" over reconfigurable circuits, have only constant memory, a shared compass, and know only their predecessor and successor along a line. Our core protocol reduces symmetry detection to equality of two binary strings laid out along a straight chain; each amoebot stores one bit of each string and participates in bit-wise arithmetic via the Primary–Secondary Circuit (PASC).

The chain is divided into blocks of length $k = \Theta(\log m)$. We set $\ell = \lceil \log_2 m \rceil + 2$, $\lambda = 2\ell$, and $k = 2^{\lceil \log_2 \lambda \rceil}$, and assume $m \geq 44$. A fully distributed procedure generates an ℓ-bit prime p with $2^{\ell-1} \leq p < 2^{\ell}$, hence $2m \leq p < 4m$. Each amoebot stores one bit of p. Primality of p is checked deterministically in parallel by testing divisibility by all $2 \leq t < m$. The prime-generation loop repeats at most $3c\ell^2$ times, runs in $O(\log^5 m)$ time, and fails with probability $\Pr[E_{\text{fail}}] \leq 1/m^c$.

For equality, we use probabilistic polynomial identity testing: evaluate the two strings as polynomials at a random point $r \bmod p$. The bits of p and r are stored across the first ℓ amoebots of every block. In parallel, block i computes its local term $s^{(i)} = a_e\, r^e \bmod p$ using modular exponentiation in $O(\log^3 m)$ time; then all $s^{(i)}$ are summed modulo p and written into the first ℓ amoebots of the chain in $O(\log^2 m)$ time. The total time for the identity test is $O(\log^4 m)$.

For testing and visualization, we additionally provide *centralized* generators for 2/3/6-fold rotations and reflection; these create controlled symmetric scenes and feed binary test instances to the distributed chain protocol without altering it.

2012 ACM Subject Classification Theory of computation → Distributed computing models; Theory of computation → Computational geometry

Keywords and phrases Amoebot model, symmetry detection, PASC, canonical tree, DFS, hex grids

1 Introduction

Symmetry is one of the simplest global properties of a shape and, at the same time, one of the hardest for a swarm of tiny, anonymous agents to agree on without centralized control. We target the **geometric amoebot model**, a stylized framework for programmable matter in which particles occupy a hexagonal grid, communicate via short "beeps" over reconfigurable local circuits, and operate with constant memory and a shared compass. Our goal is to turn the algorithmic ideas behind distributed symmetry testing into a working system in **AmoebotSim** that is faithful to the model yet practical to run and debug.

Our core design reduces symmetry detection to a **bit-string equality** problem executed along a **straight chain**. Instead of assembling and linearizing large shapes, we let each particle on the chain have **constant memory**, and we use the **Primary–Secondary Circuit**

(PASC) to assign every particle a binary index and to orchestrate bit-wise operations round by round. This lets us implement comparisons and arithmetic using only constant local state and local beeps, while keeping timing simple (synchronous rounds) and circuits bounded in degree.

For equality, we adopt the standard probabilistic reduction: view each binary string as a polynomial over a finite field and test equality by evaluating both polynomials at a random point r modulo a prime p. In our implementation, we divide the chain into blocks of length **k=O(log m)** using a PASC-driven block primitive. The bits of prime p and random value r are stored on the first $\ell = \lceil \log_2 m \rceil + 2$ particles of each block (the **parameter slice**). This allows all subsequent computations to remain purely local and bit-wise, adhering to the constant-memory constraint. The design integrates a fully distributed prime generation protocol, which is a key part of the overall algorithm and provides the necessary prime p for the subsequent polynomial identity testing.

To exercise, visualize, and stress-test the distributed chain protocol, we also provide **centralized symmetric instance generators** for 2-, 3-, and 6-fold rotational symmetry as well as reflection. These tools construct symmetric shapes (outside the chain protocol) and feed controlled binary test cases into the distributed routines; they are solely for testing and do not alter the on-chain algorithm.

The result is a modular pipeline whose stages—indexing with PASC, local bit storage, random-point evaluation, and comparison—expose clear interfaces and can be verified in isolation inside the simulator. Throughout, we adhere to constant-memory particles, simple round-based timing, and reconfigurable circuits with limited fan-out, keeping the implementation close to the theoretical model while remaining robust enough for iterative development and demos.

2 Canonical Tree Subgroup

This part of the project provides *centralized* tooling inside AmoebotSim that we used to probe and visualize symmetry. We implemented canonical root election (direction–dependent projections with a deterministic tie–break), a direction–aware DFS with Euler–tour recording, and symmetric instance generators (2/3/6–fold and reflection). These routines run in the simulator to create controlled test scenes and to debug projections/ordering; they are *not* the distributed, on–line chain protocol used elsewhere in the project. We also document the main design choices and pitfalls (global maximum selection, rotated–direction projections, position access via `Head()`, and rotation helpers such as `Rotate30` and cube coordinates).

At a high level we: (i) sample connected instances (optionally injecting centralized symmetric placements for 2/3/6–fold rotations or reflection), (ii) elect a single root for a chosen direction via a linear projection with deterministic tie–breaking, (iii) grow a canonical DFS using a fixed neighbor order (`Iterate60`), and (iv) record an Euler tour to obtain a deterministic chain per direction. These chains are used for visualization and debugging only; they are not part of the distributed chain–equality protocol.

2.1 Preliminaries

2.1.0.1 Grid and coordinates.

We use AmoebotSim's axial coordinates on the hex/triangular grid. A particle position is $(x, y) \in \mathbb{Z}^2$. The simulator exposes the twelve 30°-spaced directions via the enum `Direction`.

Table 1 Direction projections on axial coordinates (x, y) used for root election.

Direction	Projection
E	$2x + y$
ENE	$x + y$
NNE	$x + 2y$
N	y
NNW	$-x + y$
WNW	$-x$
W	$-2x - y$
WSW	$-x - y$
SSW	$-x - 2y$
S	$-y$
SSE	$x - y$
ESE	x

2.1.0.2 Neighbor order and rotations.

Given a base direction d, we traverse neighbors using `DirectionHelpers.Iterate60(d, 6, clockwise)`. Rotations use `d.Rotate30(steps)` (each step is 30°); e.g., ±3 steps give ±90°.

2.1.0.3 Projections.

To elect a root we maximize a direction–dependent linear form $ax + by$ (coefficients match AmoebotSim's geometry; see Table 1). Ties are broken deterministically using a rotated direction.

2.2 Method

2.2.0.1 Instance generation and symmetry.

We sample a connected set of lattice positions with `GenerateRandomConnectedPositions`, add particles, and—if requested—insert symmetric counterparts (2–/3–/6–fold rotations by 60° steps, or reflection across an axis). An *occupied* set prevents duplicates.

2.2.0.2 Two reference directions.

Each run fixes a base direction d (default `E`) and a second direction d' given by a fixed number of 30° steps from d (e.g., 2, 4, 6 steps).

2.2.0.3 Root election and tie–break.

We collect all candidates attaining the maximum projection in direction d, then break ties by maximizing the projection along a rotated direction $d'' = d.\texttt{Rotate30}(3 \cdot \mathit{sign})$, yielding a single root.

2.2.0.4 Canonical DFS and Euler tour.

From the root we run DFS using the ordered neighbor list from `Iterate60`. Each child stores the reverse edge as `parentDirection` $= d.\texttt{Rotate60}(3)$. We record an Euler tour (on entry and on return) to produce a deterministic chain. Live positions are read via `current.Head()`.

2.2.0.5 Symmetric placements: cube rotation and reflection.

For 120° and 60° rotations we use cube coordinates [3] (x, y, z) with $x+y+z = 0$: axial→cube via $x = q$, $z = r$, $y = -q - r$; a 60° left turn maps $(x, y, z) \mapsto (-z, -x, -y)$; cube→axial via $q' = x'$, $r' = z'$. Reflection mirrors (x, y) across the axis angle through the origin.

2.3 Parameters

The generator exposes: number of amoebots, symmetry mode (none / 2–fold / 3–fold / 6–fold / reflection), the second–direction mode, and (for reflection) the axis direction. We draw two DFS trees in different colors and log Euler chains for debugging.

2.4 Key struggles and fixes

Global maxima vs. maximum. We first (incorrectly) tried two roots; switching to one deterministic root per direction fixed DFS conflicts.
Rotated projections. Hand–written formulas were wrong; adopting the canonical table (Table 1) and ±3 steps for ±90° resolved root selection.
Position drift. Replacing manual maps with `Head()` removed stale–coordinate bugs.
`Rotate30`**.** Using `Rotate30` eliminated brittle switch–logic for rotations.
3–fold rotation. Switching to cube coordinates made 120° (and 60°) placements exact.

2.5 Status & validation

With a single root per direction, correct projection formulas, live positions, and robust rotations, canonical trees and (centralized) symmetric generators behave consistently in AmoebotSim. Visual checks confirm intended 2/3/6–fold or reflective symmetry, the elected root lies on the extreme projection, DFS is connected and acyclic, Euler tours have the expected length, and symmetric insertions do not duplicate occupied cells.

3 Centralized Prime Seed for Testing

While another subgroup was building the distributed pipeline that will supply the modulus p, our work needed a realistic p immediately to exercise the bit-level routines. We therefore implemented a *centralized stopgap* that generates a valid ℓ-bit prime inside the simulator and distributes its bits to the first ℓ particles of every block. This module can be removed later when the distributed provider of p is ready; the interface (bit placement on particles) remains the same.

3.1 Why a Local Prime?

We needed to test storage, comparison, and later broadcast assuming p is already available. The paper requires $p > 2m$ (with m the chain length). By choosing

$$\ell \;=\; \lceil \log_2 m \rceil + 2, \qquad \lambda \;=\; 2\ell, \qquad k \;=\; 2^{\lceil \log_2 \lambda \rceil},$$

any ℓ-bit integer exceeds $2m$, so a random ℓ-bit prime satisfies the bound in practice.

3.2 What We Generate

- **Prime candidate.** Sample a random ℓ-bit *odd* integer with MSB $= 1$ and LSB $= 1$.
- **Primality test.** Run a few rounds of Miller–Rabin[1]; if the candidate fails, resample. For $\ell \leq 64$ we use `ulong`; beyond that we switch to `BigInteger`.
- **Binary layout.** Keep p in *binary*. Do not centralize the integer at one node; instead, write one bit per particle across the first ℓ positions in each block of size k.

3.3 How Bits Are Placed on Particles

The initializer lays out the chain in blocks of size k and sets per-particle attributes:

- `position` $= i$: global chain index;
- `blockLeader` $\equiv (i \bmod k = 0)$: first particle of a block;
- `primeBit`: the $(i \bmod k)$-th bit of p for local positions $< \ell$;
- `primeMSB`: true exactly at the particle that stores the most significant bit of p;
- `lthamoebot`: marks the particle with local position $= \ell$;
- `rangeOfL`: true for local positions $\leq \ell$, i.e., those that participate in storing p and in the following steps.

Thus, the first ℓ particles in every block *physically hold the bits of* p. This mirrors the eventual distributed input, so downstream routines (random r, comparisons, modular arithmetic) can be exercised now without rewiring.

3.4 Guardrails and Compatibility

- **Size constraint.** With $\ell = \lceil \log_2 m \rceil + 2$, the generated p is safely $> 2m$.
- **Numerics.** Miller–Rabin trial count is a small constant (e.g., 5), sufficient for testing.
- **Swap-in later.** When the upstream module provides p, we skip local generation and write the received bits into the same attributes (`primeBit`, `primeMSB`, etc.). No other code changes.

3.5 What This Unblocked

With a realistic p present (bit-by-bit), we were able to:

1. generate candidate random bits for r across the same ℓ positions,
2. run local bit-wise routines (e.g., comparisons) to enforce $0 \leq r < p$,
3. and proceed toward the identity-testing steps that require per-block contributions.

This centralized prime was a practical bridge: faithful to the model's bit-level mechanics, but quick to implement so the rest of the pipeline could be built and tested.

4 Block Leader Election Subroutine

4.1 Goal

The goal of the **Block Leader Election subroutine** is to assign all key structural roles in a single coordinated process. Specifically, the subroutine ensures that:

- The L-th amoebot is correctly identified.
- The λ-marker position is marked.
- The block leaders are elected among the particles.
- The range from the leader to the L-th amoebot is captured.
- The source block (leader up to the first block leader) is marked.

- The east amoebots, i.e., the west neighbors of each block leader, are tagged.

Finally, the subroutine guarantees **termination** after these roles are established, so no further activity continues.

In this way, the pipeline outputs all the necessary information about markers, leaders, ranges, and block leaders in a unified process.

4.2 Introduction

The **Block Leader Election pipeline** provides all required functionality in one coordinated run. We consider a chain of amoebots arranged in a straight line, where each particle can only communicate with its immediate neighbors. The process begins by electing the **global leader**, which is the leftmost active particle in the chain. This particle first sets a marker and initiates the PASC communication pipeline. Markers are then propagated step by step: the L-th amoebot is tagged, and later a λ-marker is set. Once λ has been marked, the algorithm prepares for block leader election.

A **block leader** is elected whenever λ lies between the global leader and the leftmost still-active particle that is not the leader. At that point, all the active particles become block leaders and are highlighted. The procedure also establishes the **range of** L (from the global leader up to the L-th amoebot).

After the first block leader is identified, the algorithm performs a one-time **source block marking**, which marks all particles from the leader up to the first block leader. Next, for each remaining block leader, its west neighbor is tagged as an **east amoebot**. Finally, the eastmost block leader is unmarked to avoid duplicates, and the system reaches a clean termination.

In this way, the subroutine provides a complete structural description of the chain in one pass and ensures that no further activity continues once the roles are assigned.

4.3 Mechanism

- We consider a chain of n amoebots arranged in a line.
- Each particle can only interact with its immediate west (W) and east (E) neighbors.
- Communication is realized through *partition sets (PSETs)*, which dynamically wire pins to send or receive beeps.
- Each particle maintains local attributes (leader, markers, block leader, λ-marker, source block, east amoebot, etc.) and an integer-valued phase variable `phase`.

4.3.1 Phases 0–4: Initialization, PASC , and Feedback

The initial phases establish the leader, seed the marker pipeline, and handle passive transitions through PASC (Passive/Active State Communication) with feedback signaling.

Phase 0: Leader Election and Marker Seeding. Each particle inspects its western neighbor. The unique particle without a west neighbor identifies itself as the *global leader*. If this particle has not already been marked, it sets `isMarker = true` to seed the initial marker wave. A PASC handshake is initialized, wiring west and east neighbors into the primary and secondary partition sets.

Phase 1: PASC Send. Particles configure their pin connections and execute the *send step* of the PASC protocol, forwarding activity information towards their successors.

Phase 2: PASC Receive and Passive Transition. Each particle executes the *receive step* of PASC. If a particle becomes passive, it updates its state (`justBecamePassive = true`, `isActiveNow = false`) and sets its color to gray.

Phase 3: Feedback Setup. Particles that have just become passive attempt to notify their neighbors by sending a beep on the $FEEDBACK_PSET$. If no passive transition occurred, the feedback wiring is still set up, but no signal is emitted.

Phase 4: Feedback Processing. Particles check whether they received any feedback beeps. If they did, and they are currently carrying the marker, they attempt to forward it eastward on $MARKER_PSET$ and then unset `isMarker`. In this case, the system *advances to Phase 5*, where particles may acquire a new marker from their western neighbor. If no feedback is received, the algorithm instead transitions to the *force-forwarding path* (Phases 6–9). In either case, bookkeeping flags (`justBecamePassive`, `anyPassiveThisCycle`) are reset in preparation for the next round.

4.3.2 Phases 5–9: Marker forwarding and Latching positions

After the feedback stage, the system either continues the marker relay (Phase 5) or enters the force-forward path (Phases 6–9) to ensure that the marker progresses even if no feedback was received.

Phase 5: Marker Acquisition from the West. Particles that moved here from Phase 4 with feedback active reset `isMarker = false` and then check their western input pin. If they detect a marker signal from the west, they set `isMarker = true`, thereby relaying the marker eastwards. If not in confirm mode, this may also set a trail flag (`preResetTrail = true`) for visualization. The system then loops back to Phase 1 for continued PASC processing.

Phase 6: Forced Marker Forwarding. If no feedback was received in Phase 4, particles carrying `isMarker = true` attempt to forward it east on $MARKER_PSET$ regardless of feedback. After sending, they reset `isMarker = false`. This ensures progress even in the absence of feedback signals. The system advances to Phase 7.

Phase 7: Westward Marker Acquisition. Particles again reset `isMarker = false` and check for a signal on their west input. If received, they set `isMarker = true` and optionally record a trail if not in confirm mode. The system advances to Phase 8.

Phase 8: Second Forced Forwarding. Any particle currently holding the marker again attempts to forward it eastward on $MARKER_PSET$, then unsets `isMarker`. The system advances to Phase 9.

Phase 9: L-th and λ Latching. At this stage, particles examine whether the marker has arrived through a second hop from the west. If so, they set `isMarker = true`. Two possible outcomes are distinguished:

- *Normal run (not in confirm mode):* If a particle currently holds the marker and has not yet set `isMarker2`, it latches itself as the L-th amoebot (`isMarker2 = true`). The system then transitions to Phase 12 for bounded range marking.
- *Confirm run (after reset):* The first particle carrying the marker that has not yet set `isMarker3` latches itself as the λ-amoebot (`isMarker3 = true`), visually marked in green. A reset broadcast on $RESET_PSET$ is triggered once (`m3BroadcastDone = true`) to reactivate the system. The flag `awaitTerminateCheck = true` is set, and the system returns to Phase 1 with all particles reset to active.

4.3.3 Phases 12–13: Range of L

After the L-th amoebot has been latched as Marker2, the system performs a bounded wave to explicitly mark the interval from the global leader up to the L-th particle. This ensures that every participant in this prefix knows it belongs to the *range of L*.

In **Phase 12**, the wiring is configured through `SetupRangeOfLPC`. The rules are:

- The **leader** acts as the source, sending a signal eastward on $RANGE_OF_L_PSET$ and marking itself with `rangeOfL = true`.
- The **L-th amoebot** is a sink, listening only to its western neighbor.
- **Intermediate particles** act as relays, forwarding the signal from west to east.

In **Phase 13**, any particle that receives the signal sets `rangeOfL = true`, thereby recording its membership in the bounded region $[leader..L]$. To synchronize the system and prepare for the λ-phase, a reset is immediately broadcast on $RESET_PSET$, reactivating all nodes. The flag `inConfirmRun = true` is set, and the algorithm returns to the early phase (Phase 1), this time running in confirm mode.

4.3.4 Phases 10–11: Block Leader Election After λ

Once the λ-marker has been established, the algorithm directly transitions to block leader election. The rule is simple: as soon as the λ-position exists between the *global leader* and the *eastmost active amoebot*, **all currently active particles in that interval are declared block leaders**.

In **Phase 10**, wiring is prepared via `SetupMsetPC`, but unlike the earlier marker phases, there is no iterative propagation or repeated voting. The presence of the λ-marker is sufficient to trigger the election.

In **Phase 11**, each active particle sets `isBlockLeader = true`. The leader then emits a broadcast on $GLOBAL_MARK_PSET$ to synchronize this decision across the system and to freeze the block leader set. The system subsequently advances to Phase 20, where the eastmost block leader is unmarked.

Implication: *all currently active particles that hear the global-mark broadcast* immediately mark themselves as block leaders and jump to Phase 20. This is intentional and later refined by the eastward-wave filter (Phases 20–21).

4.3.5 Phases 20–21: Unmarking the Eastmost Block Leader

After block leader election (Phases 10–11), the system must ensure that the *eastmost block leader* does not remain marked. This is implemented to handle the corner case of uneven block size.

In **Phase 20**, a dedicated "block circuit" is established via `SetupBlockCircuit`. The mechanism routes a special pulse from the *east most amoebot* of the chain back towards the west. If a particle has no eastern neighbor, it becomes the source of this pulse, sending it westward on $SOME_NEW_PSET$. Intermediate particles relay the beep until it reaches a block leader.

In **Phase 21**, the eastmost block leader detects this pulse arriving from its eastern side. Upon receiving the signal, it unsets its block leader flag (`isBlockLeader = false`) and changes its visual color back to gray, thereby relinquishing its block leader role. Other block leaders remains unaffected. At the end of this phase, the system checks whether the *source block marking* (Phases 30–31) has already been performed. If not, it transitions to Phase 30; otherwise, it proceeds directly to Phase 22 for eastamoebot assignment.

4.3.6 Phases 30–31: Source Block Marking

After the eastmost block leader has been unmarked, the algorithm performs a one-time marking of the *source block*, defined as the segment from the global leader up to and including the *first* block leader. This ensures that exactly one contiguous prefix of the structure is identified as the origin block for further computations.

In **Phase 30**, the wiring for the source block wave is established via `SetupSourceBlockPC`. The global leader acts as the source: if it has an eastern neighbor, it sends a signal on $SOURCE_BLOCK_PSET$ and also sets `isSourceBlock = true` for itself. Intermediate particles relay the signal eastwards. However, non-leader block leaders behave as sinks: they only listen to $SOURCE_BLOCK_PSET$ from their west neighbor but do not forward it further. This guarantees that the wave terminates exactly at the first block leader.

In **Phase 31**, all particles that receive the $SOURCE_BLOCK_PSET$ signal set `isSourceBlock = true`, except for non-leader block leaders, which explicitly block propagation beyond themselves. This completes the marking of the source block. Finally, the flag `sourceBlockMarkDone = true` is set to prevent re-marking, and the algorithm advances to Phase 22 for eastamoebot assignment.

4.3.7 Phases 22–23: EastAmoebot Marking and Termination

In **Phase 22**, each remaining block leader (BL) marks the particle immediately to its west as an *eastamoebot*. This is realized by configuring a local wiring step via `SetupEastAmoebotReceivePC`, which ensures that all non-BL particles with an east neighbor listen for signals on the $EASTAMOEBOT_PSET$. Each BL that has a west neighbor then emits a beep on $EASTAMOEBOT_PSET$ towards that neighbor. As a result, the west-adjacent particle of every BL is nominated as an eastamoebot candidate.

In **Phase 23**, the actual latching and global termination occur. Any non-BL particle that receives a beep on $EASTAMOEBOT_PSET$ sets its local flag `eastamoebot = true`, thereby confirming its role as the designated eastamoebot. Immediately afterward, the system initiates a global broadcast on $FIN_TERMINATION_PSET$, which causes all particles to set `terminated = true`. This ensures a clean termination of the subroutine once the eastamoebot roles have been assigned.

In summary, the Block Leader Election subroutine performs all role assignments in a single, coordinated run: it latches the L-th amoebot and the λ-marker, elects block leaders (removing the eastmost block leader), marks the range of L and the one-time source block, tags east amoebots, and then cleanly terminates. At termination, every particle can locally answer the full API (`IsLthAmoebot`, `IsMarker3Here`, `IsBlockLeaderHere`, `InRangeOfLHere`, `InSourceBlockHere`, `IsEastAmoebotHere`), providing a complete structural summary of the chain for higher-level algorithms.

5 Prime Generation

A random prime $p \geq 2m$ is generated and broadcast from the first block to all other blocks using a distributed algorithm. A value t, defined as $t = id_{A,A_0}(A_{ik}) = ik$, is also generated with the help of the PASC algorithm and stored in the first l amoebots of every chain in time $O(\log m)$.

Afterward, the system performs $t \mid p$ division to verify whether p is prime. Once all bits of p are stored in the blocks, each block checks if p is divisible by any number t between 2 and m. This ensures that p is indeed a prime number. Each block is assigned a different t based on

its chain index, and this value is shared with all particles in the block. Once the value t is known, the block performs binary long division to check if t divides p. If $t < 1$, the prime testing is aborted; otherwise, t is incremented.

5.1 Generating a Prime and Broadcasting

Once p is generated, it must be shared with the rest of the system so that other particle blocks can assist in verifying primality. The leader block transmits the bits of p to other blocks across multiple rounds, with each block receiving and storing one bit at a time. Separate circuit IDs (partition sets) are used so that each block has its own dedicated circuit for data reception.

Round 1

Each amoebot has two flags: `blockStart` and `Marker`. All amoebots have 4 pins per edge and are arranged in a chain-like structure, as specified in the `Generate` method. In this round, each amoebot has two partition sets, connecting through the entire chain. At the beginning, all the `blockStart` amoebots (coincidentally, the `blockLeaders` of the block primitives) also have their `Marker` attribute set to `true` (all the others, `false`).
As given in the paper, the lth amoebot has its value set to 1, and for other amoebots up to the lth amoebot, they have a random value of either 0 or 1. Now, the particle that has the `Marker` attribute set (the first `blockLeader`) needs to communicate the value it holds to the `blockStart = true` particle of the other block primitives. It does so by beeping in one of the partition sets (primary or secondary). If it holds value 1, it beeps in the secondary set; otherwise, it beeps in the primary set. This exact value is then copied to the leader particles of the other block primitives that have their `Marker` attribute set to `true`.

Round 2

The amoebots are connected in a singleton chain configuration. This means that partition circuits do not extend across the entire chain. Instead, each amoebot is only connected to its immediate neighbor. For example, the first amoebot connects to the second, the second connects to the third, but the first and third are not directly connected. Inside each amoebot, there are no partition sets.
In this setup, the leader amoebots, which have their `Marker` attribute set to `true`, send a beep to their next neighbor. When this happens, the amoebot that sent the beep changes its `Marker` attribute to `false`, while the receiving amoebot switches its `Marker` attribute to `true`.
This update only affects the sender and the immediate receiver. Particles that are farther away are not affected. After this exchange, the system returns to Round 1, and the process repeats.

5.2 Generating t using PASC

The identifier of the block leader is obtained using the PASC `GetReceivedBit()` method from the SubPASC subroutine. Each block leader generates and stores its identifier, which is then propagated along the chain: if a block leader stores a bit, it connects to its successor; the successor then receives this value, stores it, and forwards it further. In this way, the identifier of the block leader is transmitted through the chain, with every amoebot that

stores a value connecting to its successor. This mechanism ensures that the id of a specific block leader (t) is stored across its subordinate amoebots.

Since the PASC algorithm itself does not include a termination condition, a separate termination check is applied. Termination occurs when there are no active beeps left, meaning no amoebot will turn passive in the next iteration. At this point, the algorithm halts. An alternative attempt was made using marker logic, similar to the approach in prime generation, where a marker would indicate which amoebot should store the leader's identifier. However, the final implementation proceeded without the marker logic.

The round-wise implementation of the above algorithm is shown below:

- **Round 1:** The pin configuration is set as per the PASC subroutine.
- **Round 2:** The `ActivateSend()` of the PASC subroutine is executed. (The `ActivateSend()` and the `ActivateReceive()` could not be in the same round because while iterating, the amoebots have to go back to the sending beep round instead of initializing it from the first).
- **Round 3:** Here `ActivateReceive()` is executed.
- **Round 4:** The PASC `GetReceivedBit()` is called to get the ID of the block leader.
- **Round 5:** A partition set is made so that the partition set connects to the successor amoebot when a `storeBit` value exists. The block leader sends its own bit ID by receiving from `GetReceivedBit()`, which is stored in `receivedValue`, which either has 1 or 0.
- **Round 6:** If a `storeBit` has a value, it goes to the next round and returns the `storeBit`, i.e., it stores the value but does not change when receiving a beep. If it is a block leader and does not have either 0 or 1, it means it does not have a stored value in the beginning. So, it stores the bit that was received from PASC and does nothing else. If it is not a block leader and does not have any stored value, it receives a beep and stores that value.
- **Round 7:** In the termination round, if there are amoebots that can turn passive in this round, they will send a beep.
- **Round 8:** All the amoebots receive this beep and then clear the pin configuration by setting it to singleton and go back to the `ActivateSend()` round to iterate. If no beep was received, then the amoebots know that there are no more amoebots that can turn passive, so they go to Round 6 and terminate.

5.3 Dividing t and p

The division was implemented in an asynchronous way where each block runs the iterations on its own. The ID of the first will be less than 2, in which case t is incremented and then the division is implemented for that block. The other blocks implement the division, get incremented, and perform the division again. The asynchronous way was then made into a synchronous pattern to find a prime number that is accepted by all the blocks.

The division was implemented with the help of the `subBinOps` subroutine where the remainder bit is taken as the result. If the result has any value in it, then it is considered a prime number for now and t is incremented, where the division is done again. Once the division is finished and the result bit for all the amoebots is set to either `true` or `false`, a `notprime` flag is set when the remainder bit of an amoebot is `false` for the entire block. If the value of this flag is found to be `true`, then it beeps through a partition set which lets the other amoebots in the chain know that the generated number was not a prime. Hence, the prime is randomly generated again and the process is repeated all over again.

If the number is found to be a prime number, it is returned at the subroutine level to be used for computation.

5.4 Difficulties Faced While Generating a Prime

- **Struggle #1:** Broadcasting the prime bits from the first block to the other blocks wasn't straightforward. Our first tries either sent them too early or didn't reach all blocks correctly.
  Solution: This was resolved by using the marker transmission subroutine after the prime generation in the source block.
- **Struggle #2:** Division in each block ran asynchronously, so synchronization became an issue.
  Solution: The blocks that completed their division and incrementation were in a loop until all the other blocks had finished. We made two circuits to run through the whole chain, which was connected in the last amoebot. The leader sends the beep, and only when the beep is received in the secondary circuit do we know that the division has completed for all the blocks in the chain.
- **Struggle #3:** Initially we thought that the lth amoebot was needed only for the first block, but later realized it was required for all the blocks.
  Solution: We again called the marker transmission subroutine so that all the other blocks would also have the lth amoebot for the binary operation.
- **Struggle #4:** After synchronization, it was difficult to check if the prime was generated or not.
  Solution: We introduced a `notprime` flag that checks if the remainder bit is `true`. Once the division is over, the amoebot with this attribute set to `true` will send a beep to indicate to the other amoebots in the chain that the generated number is not a true prime.
- **Struggle #5:** We ran into an issue where t kept incrementing all the way up to the prime itself. As a result, even when the generated number was actually prime, the division step produced a remainder of zero, causing a false negative.
  Solution: t must be incremented only up to the value of k. For this, a marker was introduced which gets sent to the next amoebot (in singleton configuration) after the current iteration of division is finished. Once the marker reaches the end of the block, it performs the final incrementation and division and then gets set to `false`, thus making the value of t increment only until $t + k$.

6 Distributed Randomness Generation and Inter-Block Communication

For the generation of a fresh random point, r, for each trial, ensuring it falls within the valid range of $0 \leq r < p$ as given in the paper [2]. This module implements a fully distributed pipeline for generating and validating r, storing it bit-wise across the particles of a chain. It also introduces a general-purpose subroutine, `SendValueToBlocks`, for propagating a value stored in one block to all other blocks in the chain so that all the blocks have the same value.

6.1 Distributed Generation of a Valid r

The process of generating the random value r is not centralized but instead occurs in parallel across all particles that form the "parameter slice" (the first l particles of each block). The procedure is designed as a multi-round state machine that ensures the final value of r is valid before allowing the main algorithm to proceed.

The core logic unfolds in these distinct phases:

- **Initial Random Sampling**: At the beginning of the procedure, each particle within the parameter slice independently generates a single random bit. Collectively, these bits form the initial candidate value for r.
- **Validation via Distributed Comparison**: The newly generated r must be strictly less than the prime p. To verify this, the algorithm invokes a distributed binary comparison subroutine (`SubBinOps.COMP`). This subroutine operates along the particle chain, examining the rBit and primeBit attributes of each particle to determine if r is greater than, less than, or equal to p.
- **Conditional Reset on Failure**: If the comparison subroutine determines that r $\geq$ p, the candidate value is deemed invalid and must be discarded. The algorithm then repeats the process of generating a new random value for r and performing the comparison. This loop continues until a valid value where r $<$ p is found. Once a valid r is accepted, the generation process for this trial concludes, and the algorithm proceeds. It is at this point that a broadcast would occur, ensuring all blocks are updated with the final, validated value of r.

The entire process is managed by a round-based state machine within each particle, ensuring all operations remain synchronized and purely local, adhering to the constraints of the amoebot model.

6.2 The `SendValueToBlocks` Subroutine for broadcast

To facilitate the reset mechanism and provide a reusable tool for other distributed tasks, we developed the `SendValueToBlocks` subroutine. Its primary function is to transmit a value present in one block and replicate it across all other blocks in the chain, ensuring all blocks are updated with the same value. This is accomplished using a two-round protocol that leverages reconfigurable circuits.

6.2.1 Key Attributes

This subsection defines the attributes that are used and what the purpose they serve for the multi-round algorithm and subroutine.

From `GenerateR.cs` The `GenerateR` algorithm, which handles the procedure for generating random r value, uses several key attributes to manage its state and the bits of the numbers it is working with:

- `rBit`: This boolean attribute is crucial as it stores the single bit of the random number r that each particle is responsible for generating and validating.
- `primeBit`: This boolean attribute stores one bit of the prime number p on each particle. It is used in conjunction with `rBit` for the bit-by-bit comparison subroutine.
- `rMSB` and `primeMSB`: These boolean attributes mark the most significant bit (MSB) of the numbers r and p respectively. They help in controlling the flow of the bit-wise comparison and other arithmetic operations.
- `lthamoebot`: A boolean attribute that marks the l-th particle in each block. This is important for defining the size of the parameter slice and for certain distributed operations.
- `blockLeader`: A boolean attribute used to mark all the block leaders in the chain.
- `rangeOfL`: A boolean attribute that marks all the amoebots starting from the block leader up to the l-th amoebot, defining the parameter slice.

These attributes collectively form the distributed representation of the numbers and control variables required for the generation and validation process.

From `SendValueToBlocks.cs` The `SendValueToBlocks` subroutine, which facilitates one-to-many communication, uses the following attributes to manage its state and data:

- `roundCounter`: An integer attribute that tracks the current round of the subroutine, ensuring that all particles remain synchronized during the two-round protocol.
- `senderBlockMarker`: A boolean attribute that identifies the single block responsible for initiating the broadcast of the value and track the progression of the marker shifting in the sender block.
- `marker`: A boolean attribute used by all other blocks to signal the beginning of the transmission phase and track the progression of the marker shifting in all the receiver blocks.
- `sendBitValue`: A boolean attribute that stores the value (0 or 1) that needs to be sent from the sender block to all the other receiver blocks.
- `finished`: A boolean attribute that indicates the subroutine has completed its execution.

These attributes allow the subroutine to perform complex, synchronized communication and data propagation across the entire chain of blocks in a decentralized manner.

6.2.2 Mechanism

- **Round 0: Value Transmission**:
  - **Pin Configuration**: All particles adopt an `AllPassive` configuration. This connects the predecessor's pins to the successor's pins, creating two long, continuous circuits along the entire chain: a primary circuit and a secondary circuit.



Figure 1 AllPassive pin configuration with amoebots marked in red being the block leaders.

  - **Sending**: The designated sender block (marked with `senderBlockMarker`) transmits the value by sending a beep on one of the two circuits. A beep on the primary partition set encodes a 1, while a beep on the secondary partition set encodes a 0.
  - **Receiving**: The designated marker particle in every other block listens on both partition sets. The circuit on which it receives a beep determines the value it stores.
- **Round 1: Marker Propagation**:
  - **Pin Configuration**: All particles switch to a `Singleton` configuration, where every pin is in its own partition set. This isolates particles from each other, allowing for targeted, direct-neighbor communication.
  - **Shifting the Marker**: The particle holding the marker (or `senderBlockMarker`) sends a beep on a specific pin connected to its successor. The successor, upon receiving this beep, sets its own marker to true. This mechanism allows the marker within each block to be shifted one position down the chain in a synchronized fashion.



Figure 2 Marker propagation where red amoebots denote block leaders, green indicates the sender marker, and blue represents the receiver marker.

This subroutine provides a robust and modular way to perform one-to-many communication and to orchestrate shifting patterns of activity across blocks, all while using only local beeps and circuit reconfiguration.

7 Distributed Modular Arithmetic and Cross-Block Accumulation

Our distributed framework not only generates and validates random values but also performs complex modular arithmetic in a purely decentralized manner. This is essential for the polynomial evaluation $f(r) = \sum_{e=0}^{m-1} a_e r^e \bmod p$, a core component of the randomized equality check. This process is orchestrated by the `ComputeF` algorithm, which utilizes several key subroutines to manage the computations across the entire amoebot chain.

7.1 The `ComputeF` Algorithm: A Multi-Phase Orchestrator

The `ComputeF` algorithm is a comprehensive, multi-round protocol that manages the entire process of calculating the polynomial $f(r)$. It's designed as a state machine where each particle locally tracks the current round to remain synchronized with its neighbors. The process unfolds in three main phases: an initial setup, a looping phase for modular exponentiation, and a final phase for cross-block summation.

- **Phase 1: Initialization**
  The process begins with a simple setup. The amoebots establish a direct, passive circuit connection across the entire chain. An activation marker, `aMarker`, propagates along the chain to signal the start of computation; once a particle receives it, the system transitions to the core computational phase. For example, if `aMarker` is at the 8th amoebot, the e bits are stored least significant to most significant within that block: the `eValue` recorded across the first four amoebots is `0001`, and `eMSB` is set to `true` on the 4th amoebot.
- **Phase 2: Modular Exponentiation and Multiplication**
  This is the main computational loop. It's dedicated to evaluating the individual terms $a_e r^e \bmod p$ for each block. At the beginning of each iteration, the algorithm calls the `SubComputeRmodP` subroutine to calculate $r^e \bmod p$ (stored as `power`). It then uses the `BinaryOps.MUL` subroutine to compute `result = result x power`. After each multiplication, it performs a modular reduction $result \leftarrow result \bmod p$ to keep the numbers within a manageable range so that it can be stored within a block and doesn't overflow. This loop continues, iterating over each bit of the a-string, until all the bits of a have been processed. It also has an inner loop with the `eMarker` which tracks the `eValue` and goes up until the eMSB.
- **Phase 3: Cross-Block Summation and Finalization**
  Once the modular exponentiation for a specific block is complete, the individual $s(i)$ values (representing the term for that block) are ready to be summed up. The algorithm utilizes the `PASC` subroutine to facilitate a tree-like accumulation process, where adjacent blocks add their $s(i)$ values together. This is a distributed, hierarchical sum that eventually converges the final result to the head of the chain. After the sum is completed, the algorithm sets the pin configuration to a `TerminationConfig` to signal to all amoebots that the algorithm is finished and we have $f(r)$ for the entire chain ready.

7.2 The `SubComputeRmodP` Subroutine

The `SubComputeRmodP` subroutine is a vital component for the modular exponentiation required by `ComputeF`. It is responsible for efficiently calculating the $r^e \bmod p$ term for each



Figure 3 Termination pin configuration wherein it is like the AllPassiveConfig except the last amoebot has a single partition set connecting to it's predecessor SecPset with amoebots marked in red being the block leaders.

block in a distributed manner and then combining all the computed $s(i)$ values together to finally form $f(r)$ for the whole chain. This subroutine itself is a state machine that orchestrates a series of bit-wise arithmetic operations. It leverages other subroutines like `SubDivision2`, `SendValueToBlocks`, `BinaryOps.MSB`, `BinaryOps.MULT` and `BinaryOps.ADD` to perform the calculations. The output of this subroutine, the `result` value, is then passed back to the `ComputeF` algorithm to be used in the next step of the polynomial evaluation.

7.2.1 Key Attributes

This subsection defines the attributes that are used and the purposes they serve for the multi-round algorithm and subroutine.

From `SubComputeRmodP.cs` The `SubComputeRmodP` subroutine uses several key attributes to manage its state and the bits of the numbers it is working with:

- `eValue`: This boolean attribute stores a single bit of e.
- `eMSB`: This boolean attribute marks the most significant bit (MSB) of the e.
- `eMarker`: A boolean attribute used to mark the current bit of the e being processed within a block.
- `aMarker`: A boolean attribute that is used to mark the current block being processed, and is shifted from block to block.
- `rValue`: This boolean attribute stores a single bit of the random number r.
- `pvalue`: This boolean attribute stores a single bit of the prime number p.
- `result`: This is a boolean attribute that stores a bit of the intermediate and final results of the computations, such as `result = result * power`.
- `power`: This boolean attribute stores a bit of the intermediate and final power values, such as `power = r^e`.
- `isSenderBlock`: A boolean attribute that marks the entire block that is designated to send its `s(i)` value to predecessor block during the summation phase.
- `isSenderBlockLeader`: A boolean attribute that specifically marks the block leader of the sender block.
- `isReceiverBlock`: A boolean attribute that marks the entire block that is designated to receive an `s(i)` value from a sender block(which will be the successor block if present) and add it to its own value.
- `sum`: This attribute is crucial for storing the partial sum `s(i)` for each block. After all individual block computations are complete, this same attribute is used to store the final polynomial value `f(r)` after summing up all the `s(i)` values from all the blocks.

7.2.2 Mechanism

The `SubComputeRmodP` subroutine is an intricate, multi-round state machine designed to perform modular exponentiation $r^e \pmod p$ and then combining all the computed $s(i)$ values together to finally form $f(r)$ for the whole chain. in a distributed fashion. The process is

synchronized across all amoebots and progresses through a series of distinct rounds, each with a specific purpose.

- **Rounds -2 to -1: Initialization and Broadcast**
  The subroutine begins with a crucial setup phase. All amoebots configure their pins for an `AllPassive` state, effectively creating a single, continuous circuit that spans the entire chain. A designated `aMarker` bit is then broadcast to all other amoebots. This synchronized, broadcast signal ensures that all blocks in the chain are aware of the beginning of the computational phase and can proceed in unison. If an amoebot receives a beep on the primary partition set, it knows to continue; otherwise, if a beep is received on the secondary partition set, it signifies a termination condition, and the process jumps to Round 25.
- **Rounds 0 to 3: Setting Up the First Term**
  The process of calculating the initial modular power, `power = r mod p`, is set up using the `SubDivision2` subroutine. Once this operation is finished, the remainder is retrieved and set as the initial `power` value. For the block leader, the `result` bit and its corresponding `result MSB` are set to 1, initiating the multiplicative process. The MSB of the `power` value is then determined using a `binOps` MSB operation.
- **Rounds 4 to 5: Broadcasting the Exponent Bit**
  The core exponentiation loop begins with a broadcast of the `eMarker` bit, which represents a single bit of e. This is done using an `AllPassiveConfig` where the amoebot with the `eMarker` sends a beep on either the primary partition set (if the bit is 1) or the secondary partition set (if the bit is 0). All other amoebots receive this beep and update their `eMarker` bit accordingly, ensuring the entire chain is synchronized with the current bit of the e being processed.
- **Rounds 6 to 17: The Core Exponentiation Loop**
  Based on the value of the `eMarker` bit, the subroutine performs a series of bit-wise arithmetic operations. If the `eMarker` is 1, it triggers a chain of operations: `result = result x power`, followed by a modular reduction `result = result mod p`. Regardless of the `eMarker` value, the subroutine also always computes `power = power x power` and then `power = power mod p`. This repeated squaring is the core of the modular exponentiation algorithm. This process continues through several rounds of activation and reception for the multiplication and division subroutines.
- **Rounds 18 to 19: Shifting the `eMarker`**
  To move to the next bit of the `eValue` bit, the `eMarker` is shifted one position down the chain. This is accomplished using a `Singleton` pin configuration that allows for direct, neighbor-to-neighbor communication. The particle holding the `eMarker` sends a beep to its successor, which in turn becomes the new `eMarker`. Once the marker has shifted, all amoebots transition back to an `AllPassiveConfig` to check if an `eMarker` is still present. If so, the loop repeats from Round 4; if not, the exponentiation for the current block is complete.
- **Rounds 20 to 25: Accumulating the Result**
  Upon exiting the exponentiation loop, the intermediate `result` is added to a running `sum` (`sum = sum + result`) followed by a modular reduction (`sum = sum mod p`) to prevent overflow. These operations are performed using the `binOps.ADD` and `binOps.DIV` subroutines, ensuring the final sum for the current block is correct and within the valid range.
- **Rounds 26 to 28: Shifting the Main Marker and Finalizing**
  The `aMarker` is shifted to the next block, signaling that the current block's computation

is complete. Amoebots check for the presence of the `aMarker` across the chain. If it's still present, the process repeats from Round –2 for the next block. If no `aMarker` is found, it signifies that all blocks have finished their individual calculations.

- **Rounds 29 to 33: Cross-Block Summation and `eValue` Increment**
  Once all blocks have computed their individual terms, the `PASC` and `SendValueToBlocks` subroutines are used in a tree-like accumulation process. Concurrently, if a new block is about to be processed, the `eValue` is incremented by 1 using `binOps.ADD` to prepare for the next term of the polynomial.
- **Rounds 34 to 45: Cross-Block Summation and Termination**
  The cross-block summation begins with the `PASC` subroutine, followed by the `SendValueToBlocks` subroutine to transfer the partial sums from sender blocks to receiver blocks. The receiver blocks then use `binOps.ADD` to sum the incoming value with their own. This process continues until the final polynomial value is converged to the first amoebot in the chain. The subroutine finally terminates when a series of `TerminationConfig` checks confirm that all blocks have completed the summation.

7.3 The `SubDivision2` Subroutine: Handling Edge Cases in Division

During modular arithmetic, a common challenge arises when performing a division where the dividend is smaller than the divisor. Standard division subroutines, such as `binOps.DIV`, are not designed to handle this specific case and may produce an incorrect result. To address this, we developed the `SubDivision2` subroutine.

This subroutine first performs a distributed comparison between the dividend and the divisor using `SubBinOps.COMP`. It then proceeds with a conditional logic:

- If the dividend is found to be greater than or equal to the divisor, the subroutine calls the standard `binOps.DIV` to perform the modular division.
- If the dividend is found to be less than the divisor, `SubDivision2` recognizes this special case and correctly returns the value of the dividend as the remainder without performing any further division.

This small but crucial modification ensures the correctness of all modular reduction operations within the `ComputeF` algorithm, regardless of the size of the operands.

8 Orchestrating the End-to-End Symmetry Check

This section describes how we connect the available subroutines into one end-to-end run that decides whether two bit strings (stored on a single simple chain) are equal (symmetric) via a randomized polynomial identity test (PIT). We keep the integration lean: we operate on a *single* chain, run $T = \lceil \log m \rceil$ trials (with c=1), and ignore the paper's lower bound $m \geq 44$. Both the prime p and the random points r are supplied by *distributed* subroutines that write their bits directly onto particles.

8.0.0.1 Chain placement and user-controlled strings.

We begin by placing a straight chain A of length m. Each particle $i \in \{0, \ldots, m-1\}$ stores two local bits `aBit`$[i]$ and `bBit`$[i]$ representing strings $a, b \in \{0,1\}^m$. A UI flag in the simulator `forceEqualAB` lets the user decide whether $a = b$ at initialization: if `forceEqualAB` is true, we first sample a uniformly at random and then set $b \leftarrow a$; otherwise we draw a and b independently at random (one bit per particle). This provides controlled test cases (equal vs. non-equal) for the distributed checker.

8.0.0.2 Distributed inputs.

At the start of the run, a distributed prime-generation subroutine produces an ℓ-bit prime p (bit-wise) and writes it to the parameter slice of *every* block. At the beginning of every trial $t \in \{1, \ldots, T\}$, a distributed randomness subroutine produces a fresh r with $0 \le r < p$, again written (bit-wise) to each parameter slice. All values live in binary on particles; no centralized integers are kept.

8.0.0.3 Trial loop and ownership of exponents.

Fix a trial's pair (p, r). The polynomial of a bit string x is $f_x(t) = \sum_{e=0}^{m-1} x_e\, t^e \bmod p$. Each block C_i owns exponents $e = i \cdot k + j$ for $j = 0, \ldots, k-1$. Blocks run k *local rounds*; in local round j, block C_i handles $e = i \cdot k + j$.

8.0.0.4 Per-block contribution and accumulation.

For $x \in \{a, b\}$ and its current exponent e, block C_i computes

$$s_x^{(i,j)} \;=\; x_e \cdot r^e \pmod p,$$

using bit-wise right-to-left modular exponentiation and modular multiplication (all over the ℓ-bit parameter slice). Then a modulo-p *cross-block accumulation* adds the single ℓ-bit addend from each block into a global running total $F_x \in [0, p-1]$ stored on the first ℓ particles of the chain. After k local rounds, $F_x = f_x(r)$.

8.0.0.5 Decision and repetition.

At the end of the trial we compare F_a and F_b (bit-wise) on the first ℓ particles. If $F_a \neq F_b$, we output NotEqual and stop early. Otherwise we start a new trial with a fresh distributed r. After $T = \lceil \log m \rceil$ successful trials (all equal), we output Equal. This follows the standard PIT guarantee: with independent r values, the false-equality probability decays exponentially in T.

Run–level description

We run the end-to-end check on a single straight chain of m particles. A flag selection `forceEqualAB` in the simulator controls whether the two strings are identical by construction.

Parameters. We set $\ell = \lceil \log_2 m \rceil + 2$, $\lambda = 2\ell$, and $k = 2^{\lceil \log_2 \lambda \rceil}$. The number of independent trials is $T = \lceil \log m \rceil$ (we take $c = 1$).

Step 0: place bits on the chain. We place a straight chain of length m. Each particle i stores two input bits, $a[i]$ and $b[i]$. If `forceEqualAB` is true we set $b = a$; otherwise we draw b independently at random. Thus each particle physically holds one bit of each string.

Step 1: block primitive. We partition the chain into consecutive blocks C_i of size k and mark, in every block, the first ℓ particles as the "parameter slice". These ℓ positions are reserved to store parameters shared by the block.

Step 2: distributed prime. Using the prime-generation subroutine, the system produces a prime p with ℓ bits (and thus $p > 2m$ in practice). The ℓ bits of p are written, in binary, into the parameter slice of every block.

Step 3: T randomized trials. For $t = 1, \ldots, T$ we repeat:

- Generate a fresh random r with $0 \le r < p$ using the distributed routine, and write its ℓ bits into each parameter slice.
- Initialize two running sums F_a and F_b (modulo p). Then, in parallel across all blocks, each particle at global index e contributes $r^e \bmod p$ to F_a if $a[e] = 1$ and 0 otherwise; similarly for F_b using $b[e]$. Contributions are accumulated with the modulo-p addition subroutine so that, at the end of the round, the chain holds $F_a = \sum_e a[e] \cdot r^e \bmod p$ and $F_b = \sum_e b[e] \cdot r^e \bmod p$.
- Compare F_a and F_b bit-wise (distributed equality). If they differ, the run terminates immediately with the verdict "not equal".

Step 4: decision. If all T trials report $F_a = F_b$, we output "equal". By the standard fingerprinting bound, the error probability is at most $1/p$ per trial; with $T = \lceil \log m \rceil$ the overall error becomes negligible for our parameter sizes.

8.0.0.6 Outcome.

With these components, each trial evaluates $f_a(r)$ and $f_b(r)$ in parallel across blocks, aggregates the results in place on the chain, and compares them locally. Repeating for $\lceil \log m \rceil$ fresh r values yields a practical, fully distributed equality test consistent with the theoretical pipeline.

9 Contributions

9.1 Canonical trees and symmetric instance generator

Ahmet Can Kaytaz, Ibrahim Ali Khan, Thevin Arokiaraj Lawrence

9.2 Centralized prime seed for testing

Ibrahim Ali Khan, Ahmet Can Kaytaz

9.3 Block primitives Election & tagging

Pooja Garg

9.4 Marking of λ, L-th Amoebot, Source Block, East Amoebot, and Range of L

Pooja Garg

9.5 Prime Generation and Broadcasting

Thevin Arokiaraj Lawrence, Rasika Jayavelu

9.6 Block ID Computation

Thevin Arokiaraj Lawrence, Rasika Jayavelu

9.7 Generate r and implementing SendValueToBlocks subroutine for broadcast

Ibrahim Ali Khan

9.8 Calculating $s(i) = a_e r^e \pmod p$ using modular exponentiation

Ibrahim Ali Khan

9.9 Calculating $f(r)$ after summing up all $s(i)$ from all the blocks

Ibrahim Ali Khan

9.10 End–to–end orchestration

Ahmet Can Kaytaz

9.11 Documentation

Ahmet Can Kaytaz (lead drafting), Pooja Garg, Ibrahim Ali Khan, Rasika Jayavelu, Thevin Arokiaraj Lawrence

References

1 Gary L. Miller, Michael O. Rabin. Miller–rabin primality test. `https://en.wikipedia.org/wiki/Miller%E2%80%93Rabin_primality_test`, 2025. Accessed: 2025-07-XX.

2 Andreas Padalkin, Christian Scheideler, and Daniel Warner. The structural power of reconfigurable circuits in the amoebot model. *Natural Computing*, 23(4):603–625, 2024.

3 Amit J. Patel. Hexagonal grids. `https://www.redblobgames.com/grids/hexagons/`, 2014. Accessed: 2025-07-XX.